



Departamentul Automatică și Informatică Industrială

**Facultatea Automatică și Calculatoare
Universitatea Națională de Știință și Tehnologie
POLITEHNICA din București**

Splaiul Independenței 313, 060042, București, România
Sala ED 412, Tel. 021/402.92.69
www.aii.pub.ro, email: secretariat.aii@upb.ro



Sesiunea de Comunicări Științifice Studențești

Ediția 2025

Sistem pentru detecția și raportarea evenimentelor de securitate în rețele industriale de comunicații.

**Autor - Theodor Predescu, Automatică și Calculatoare, specializarea Ingineria
Sistemelor, anul 4, grupa 342 A1**

Adresă mail - predescutheodor@gmail.com

Îndrumător - Conf. dr. Ing. Florin Daniel Anton

Cuprins:

1. Introducere	<u>pag. x</u>
2. Prezentarea domeniului din care face parte lucrarea	<u>pag. y</u>
3. Descrierea problemei si prezentarea solutiei propuse	<u>pag. z</u>
4. Prezentarea aplicatiei.....	
5. Concluzii si planuri de viitor	
6. Anexe (dacă este cazul)	
7. Bibliografie	

Introducere.....	3
Prezentarea domeniului.....	3
Descrierea problemei și soluția propusă.....	4
Prezentarea aplicației.....	5
Detalii de implementare.....	5
Componenta de filtrare la nivel kernel	6
Componenta de intermediere.....	6
Componenta centrală.....	7
Componenta de administrare	7
Experiența utilizatorului	7

Introducere

Trăim într-o perioadă în care, datorită progresului tehnologic cultural, oamenilor le este dată oportunitatea de a se exprima artistic și inovativ. Aceste forme de exprimare curente includ atât domenii consacrate de-a lungul timpului, precum pictura, poezia sau sculptura, cât și domenii moderne, dezvoltate în ultimele decenii, precum cinematografia, fotografia, alături de domenii cu un caracter tehnic: robotica, automatizarea, dezvoltarea aplicațiilor software și a jocurilor video.

De la aplicații software populare, precum Minecraft, până la sisteme complexe de operare, precum Linux, numeroase proiecte demonstrează impactul semnificativ pe care îl pot avea inițiativele individuale sau colaborative asupra dezvoltării tehnologice. Aceste exemple evidențiază faptul că inovația poate apărea într-o varietate de contexte și poate conduce la soluții utilizate la scară largă.

Odată cu avansarea tehnologică, a apărut de asemenea nevoia de protejare împotriva factorilor externi ce ar perturba fluxul de lucru, iar în domenii ce au componente de tip software, această problemă este amplificată.

Prezentarea domeniului

Ca urmare a avansării în complexitate a unui ecosistemul digital ce este în continua extindere, securitatea aplicațiilor și a serviciilor devine tot mai dificilă și mai importantă. Pentru a funcționa în mod eficient și a fi utilizate la capacitate maximă, aceste servicii necesită un nivel ridicat de securitate, capabil să protejeze împotriva numeroaselor tipuri de atacuri. Serviciile de securitate sunt supuse unor cerințe în continuă creștere, dorindu-se adaptarea atât

la complexitatea amenințărilor, cât și la volumul tot mai mare de trafic generat de creșterea numărului de utilizatori și a dispozitive conectate.

Un serviciu de securitate are rolul de a proteja sistemele împotriva acțiunilor nedorite și a actorilor malițioși, prin identificarea și mitigarea vulnerabilităților existente. În acest context, mecanismele de protecție trebuie să funcționeze în mod riguros și continuu, acoperind un spectru cât mai larg de posibile puncte slabe. În schimb, compromiterea sistemului poate fi realizată prin exploatarea unei singure vulnerabilități neidentificate sau tratate insuficient.

Această realitate evidențiază complexitatea ridicată a proceselor de securizare, care necesită soluții eficiente, adaptabile și capabile să răspundă în timp real la evoluția amenințărilor.

Descrierea problemei și soluția propusă

O cerință esențială în mediul industrial reprezintă monitorizarea dispozitivelor în cadrul unei rețele locale. Se dorește un nivel cât mai mare de control asupra conexiunilor ce pot avea loc între dispozitive, mai ales dacă aceste conexiuni sunt externe rețelei.

Monitorizarea dispozitivelor în cadrul unei rețele implică analiza detaliată a pachetelor de date și identificarea comportamentelor anormale sau care nu se conformează cu politicile de comunicare stabilite.

Soluția propusă în această lucrare oferă un răspuns problemei descrise prin proiectarea și implementarea unui sistem integrat de monitorizare și control a traficului în rețea, bazat pe o arhitectură modernă. Sistemul are la bază mecanisme de inspecție și filtrare la nivel kernel, specifice sistemelor de operare de tip Linux. Soluția presupune interceptarea și preluarea informațiilor de interes a pachetelor de date în puncte critice ale fluxului acestora, urmată de o filtrare eficientă și rapidă a acestora. Informații despre pachetele analizate sunt apoi transmise unei unități centrale pentru o inspecție generală.

Filtrarea acestora se realizează pe baza unei liste definite de utilizator numită, care specifică atât adresele cu care este permisă comunicarea, cât și porturile prin care aceasta se poate desfășura.

Modul de funcționare a aplicației este reprezentat de unul din cele 4 profile disponibile:

- LISTENING – aplicația rulează în mod permisiv, acceptând conexiuni nepermise în listă, dar sunt raportate unității centrale

- **REACTIVE** – rulează în mod strict, refuzând conexiunile ce nu sunt permise și le raportează mai departe
- **DISABLED** – aplicația permite conexiuni ce nu sunt definite, iar raportarea nu se face
- **MONITORING** – aplicația este în procesul de adăugare automată a conexiunilor în lista de conexiuni permise (acest mod de funcționare este folosit pentru definirea accesului în mod dinamic).

Soluția propusă cuprinde o aplicație desktop ce va fi în strânsă comunicare cu unitatea centrală de stocare a informațiilor. Pe baza informațiilor preluate, un utilizator poate avea o imagine de ansamblu a traficului produs în rețea și a regulilor de funcționare a fiecărui dispozitiv. Pe baza acestor informații conexiunile inițiate sunt diferențiate. Aplicația face ușor de depistat dispozitivele ce încep o comunicare ce se abate de la profilele setate în rețea, indiferent dacă comunicarea a fost blocată sau permisă. De asemenea, aplicația facilitează modificarea dinamică a profilelor dispozitivelor din rețea, atât a modului de lucru cât și a modificării dispozitivelor permise.

Prezentarea aplicației

Detalii de implementare

Aplicația este formată dintr-un număr de componente logice, fiecare având un scop bine definit. Aceste componente sunt:

1. **componenta de filtrare la nivel kernel**, responsabil de interceptarea pachetelor, extragerea informațiilor relevante despre acestea și transmiterea acestora, și blocarea pachetelor menționate în fișierul de configurații.
2. **componenta de intermediere**, responsabil de intermedierea informațiilor între modulul kernel și serverul. Acesta de asemenea este responsabil și de citirea fișierelor de configurații, sincronizarea acestora cu informațiile primite de la server și trimiterea noilor dispozitive ce trebuiesc blocate către modul.
3. **componenta centrală**, responsabil de stocarea tuturor informațiilor primite de la dispozitivele monitorizate și intermedierea legăturii între totalitatea dispozitivelor și aplicația adminului.

4. **componenta de administrare**, ce ofera posibilitatea de vizualizare a conexiunilor ce au avut loc pe dispozitivele monitorizare si modificarea configuratiilor acestora intr-un mod individual sau global.

Componenta de filtrare la nivel kernel

Aceasta componenta reprezinta un modul kernel scris in limbajul de programare C ce ruleaza in spatiul adminului unitatii. Principala caracteristică a acestuia consta în definirea și înregistrarea unui mecanism de interceptare a pachetelor (network hook) utilizand frameworkul '*Netfilter*', care permite interceptarea, analiza si blocarea pachetelor in retea, in anumite puncte specifice ale procesului de procesare. Cu ajutorul acestui framework este registrata pentru reapelare functia '*packet_hook*' la intrarea pachetelor în procesarea locala ('*NF_INET_LOCAL_IN*').

Componenta de intermediere

Aceasta componenta reprezinta un agent local, 'daemon', ce ruleaza un spatiul utilizatorului. Limbajul ales pentru crearea acestei componente a fost C++, pentru a utiliza componente cu un grad de abstractizare mai mare, pastrand totodata puterea si viteza de procesare a limbajului C. Agentul este responsabil de trimiterea informatiilor ce au fost extrase din pachete in modulul kernel spre unitatii centrale, si de asemenea de sincronizarea setarilor si regulilor noi selectate in aplicatie cu regulile si modul de functionare a componentei de filtrare.

Programul comunica cu modulul kernel este facuta prin intermediul unui Miscellaneous character device, ce ofera posibilitatea folosirii unui character device intr-un mod simplificat. Modulul kernel trimite informatii referitoare la pachetele primite intr-un character device, informatii ce sunt periodic citite de agentul local si trimise in grupari catre componenta centrala.

Intr-un mod similar, programul trimite periodic mesaje catre server pentru a verifica daca configurarea aplicatiei trebuie modificata. In cazul in care serverul il informa ca modificari au avut loc, acesta va scrie in fisierul de tip character device pentru a modifica modul de functionare a componentei de filtrare.

De asemenea, componenta este responsabila de stocarea locala a configuratiei primite de la server, cat si de citirea acesteia la inceputul rularii programului. Aceasta stocare se face in 3 fisiere, responsabile pentru stocarea unei indexari unice a dispozitivului, salvarea modului de rulare a aplicatiei, a locatiei fisierului ce contine lista de conexiuni blocate si informatii despre conexiunea cu serverul (adresa ip si portul ce trebuie utilizat), si fisierul responsabil de retinerea dispozitivelor ce trebuiesc blocate.

Componenta centrală

Componenta centrala reprezinta serverul aplicatiei responsabila de gestionarea legaturii dintre dispozitivele utilizate si aplicatia desktop. Aceasta este responsabila de centralizarea informatiilor transmise de modulele kernel ce ruleaza pe dispozitivele in legatura directa cu acesta, si reformatarea acesteia pentru a putea fi vizualizata si prelucrata de aplicatia utilizatorului admin. Totodata, prin componenta centrala, noile reguli setate din aplicatia desktop sunt transmise catre dispozitive.

Componenta de administrare

Componenta de administrare este reprezentata de aplicatia desktop ce ofera flexibilitate utilizatorilor pentru control asupra dispozitivelor asociate serverului. Prin aceasta un utilizator poate modifica atat modul de rulare, cat si reguli de blocare individuala a conexiunilor externe ce au fost, sau pot fi in legatura cu dispozitivele. De asemenea, ofera posibilitatea de vizualizare de tip graf a conexiunilor care au fost facute cu dispozitivele suport.

Experiența utilizatorului

Pentru utilizarea aplicației pe dispozitivele vizate pentru monitorizare, sunt necesari 4 pași:

1. **Crearea fișierelor de configurare**, proces realizat prin definirea fișierului de configurare și de definire a conexiunilor dorite. Pentru configurare, aplicația va căuta

fișierul „*mymodule.conf*”, în locația „*/etc/*”. În acest fișier va fi definită adresa de ip locală a serverului căruia îi va trimite informații, portul pe care acesta va trebui să în folosească în comunicarea cu serverul, locația fișierului ce va fi utilizat în definirea conexiunilor permise și modul de funcționare a aplicației pe dispozitivul suport.

O formă generală a acestui fișier este:

```
# Pentru a comenta o linie, se poate folosi
# caracterul „#”

# exemplu de adresă - 192.168.0.112
server IP <adresa ip a serverului>

# portul trebuie regăsit în valoare numerică
server port <port>

# se dorește locația globală a fișierului
allowed file <locația fișierului de definiții>

<mod>
# Modul de rulare este procesat fără diferențiere între
# majuscule și minuscule.
# se acceptă - listening, Monitoring, REACTIVE, DiSaBled
```

2. **Compilarea modului kernel** se realizează prin rularea comenzii „*make*”. Această comandă utilizează fișierul de configurare Makefile:

```
obj-m += kernel_module.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)

all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

3. **Adăugarea modului în spațiul kernel** realizată prin rularea comenzii „*sudo insmod kernel_module.ko*”.
4. **Rularea daemonului** se face prin executarea binarului cu privilegii elevate: „*sudo ./daemon*”.

Aplicația desktop va porni din fereastra de

